



Department of Computer Engineering

Bu-Ali Sina University

Revolutionary Computing Course

Revolutionary Computing 1st Assignment Travelling Salesman Problem using Evolutionary Algorithms

By:

Alireza Maleki

Course Professor:

Professor Hassan Khotanlou

Spring 2023-24

TABLE OF CONTENTS

1	Introduction	1
2	Material & Methodology	1
2.1	Representation	1
2.1.1	Initializing the Population	1
2.2	Parent & Survival Selection	2
2.2.1	Linear-Ranking Selection	2
2.2.2	Fitness-Proportionate Selection	2
2.2.3	Roulette Wheel Selection	3
2.3	Crossover	4
2.3.1	Partially Mapped Crossover (PMX)	4
2.3.2	Edge Recombination Crossover (ERO)	5
2.4	Mutation	5
2.4.1	Swap Mutation	6
2.4.2	Insert Mutation	6
2.5	Dataset	7
3	Experimental Results	7
3.1	Results	8
3.2	Experiments	9
4	Conclusion	10
	References	11

1 INTRODUCTION

The Traveling Salesman Problem (TSP) is a classic combinatorial optimization problem where the goal is to find the shortest route visiting each city in a given list exactly once and returning to the starting city. It is an NP-hard problem and evolutionary algorithms are commonly used to find good approximate solutions.

In this report, an evolutionary algorithm is implemented to solve a 127-city TSP instance provided in the *TSPDATA.txt* file. Two recombination methods (edge recombination and partially mapped crossover) and two mutation methods (swap mutation and insert mutation) are tested. Additionally, two selection methods (roulette wheel and linear ranking) are evaluated. The algorithm is run 1000 times for each method combination and the best tour length and number of iterations to convergence are recorded.

2 MATERIAL & METHODOLOGY

2.1 Representation

The Traveling Salesman Problem (TSP) is a classic optimization problem that involves finding the shortest possible route for a salesman to visit a set of cities and return to the starting point. Evolutionary algorithms (EAs) have emerged as powerful tools for solving complex optimization problems, including the TSP. An essential component of an EA is its representation, which defines how solutions to the problem are encoded and manipulated.

2.1.1 Initializing the Population

In this assignment, we will employ a permutation representation for encoding TSP solutions. A permutation is simply an ordered arrangement of the city indices, where each index corresponds to a unique city. This representation is well-suited for the TSP as it directly captures the ordering of the cities, which is crucial for determining the overall route length. To represent a solution using a permutation, we initialize a list of city indices, where each index represents a city. The initial order of the indices defines the starting route for the salesman. During the EA's search process, the permutation representation allows for efficient manipulation of the route by swapping or inserting city indices. This flexibility is essential for the EA to explore the search space and find better solutions. In this assignment, we considered the initialized population's size with 100. Every individual in the population was randomly shuffled through the cities and produced. Note that each individual con-

tains a number of 127 cities. So, Our first generation includes 100 presentations from 127 cities. The choice of representation can significantly impact the performance of an EA. The permutation representation is particularly suitable for the TSP due to its direct connection to the problem's structure and its ability to facilitate efficient search operations.

2.2 Parent & Survival Selection

Parent selection is a crucial step in evolutionary algorithms (EAs), as it determines which individuals will be used to generate offspring. The selection process aims to identify individuals with higher fitness, as they are more likely to produce offspring that are also fitter. This allows the EA to gradually improve the population over generations.

There are two main approaches to parent selection: linear-ranking selection and fitness proportionate selection.

2.2.1 Linear-Ranking Selection

In linear-ranking selection[1], each individual is assigned a fitness rank. The ranks are assigned based on fitness, with higher-fitness individuals receiving higher ranks. The selection probability of an individual is then proportional to its rank. This means that higher-fitness individuals have a higher chance of being selected as parents, while lower-fitness individuals have a lower chance as the concept is comprehensible from the below formula.

$$P_{lin-rank} = \frac{2 - s}{\mu} + \frac{2i * (s - 1)}{\mu(\mu - 1)} \quad (1)$$

Linear-ranking selection is effective in maintaining diversity in the population, as it gives lower-fitness individuals a chance to reproduce even if they are not the fittest. This can help to prevent premature convergence, where the EA stops improving because it has converged on a local optimum.

However, linear-ranking selection can also lead to problems if the fitness range of the individuals is very large. In this case, the fitness ranks may be spread out over a wide range, which can make the selection probability of lower-fitness individuals very small. This can hinder the EA's ability to find the best solution.

2.2.2 Fitness-Proportionate Selection

Fitness proportionate selection (FPS)[2], also known as roulette wheel selection, is a more direct method of selecting parents based on fitness. In this method, each individual is assigned a prob-

ability of being selected that is proportional to its fitness which is obvious in the FPS formula. This means that higher-fitness individuals have a higher probability of being selected, while lower-fitness individuals have a lower probability.

$$P_{FPS} = f_i / \sum_{j=1}^{\mu} f_j \quad (2)$$

Fitness proportionate selection is efficient in terms of computational resources, as it only requires a single pass through the population to select parents. However, it can suffer from the same problem as linear-ranking selection if the fitness range of the individuals is very large.

2.2.3 Roulette Wheel Selection

Roulette wheel selection[3] is a popular method for selecting parents in evolutionary algorithms (EAs). It is a simple and efficient method that is well-suited for binary representations, such as the permutation representation used in this assignment.

In this assignment, a crucial task involves computing probabilities for each of these individuals using linear ranking and FPS methods and subsequently transferring them to the Roulette wheel selection algorithm. In roulette wheel selection, each individual in the population is assigned a probability of being selected. The probability is proportional to the individual's fitness, with higher-fitness individuals having a higher probability of being selected. This means that the selection process is biased towards fitter individuals, which helps to ensure that the population is gradually improved over generations.

To implement roulette wheel selection, a roulette wheel is constructed. The wheel is divided into slices, with the size of each slice proportional to the fitness of the corresponding individual. During the selection process, a random number is generated, and the individual corresponding to the slice where the roulette wheel stops is selected as a parent.

Roulette wheel selection is a simple and efficient method that is easy to implement and can be effective for a wide range of problems. However, it can suffer from a problem called early convergence. This is where the EA becomes stuck in a local optimum, where the fitness of the population is not improving.

This can be a problem for the traveling salesman problem (TSP), as there are a vast number of local optima. To combat this problem, we can use a different parent selection method, such as linear-ranking selection, which can help to maintain diversity in the population and prevent early convergence which is used to pass the probabilities to Roulette Wheel selection.

2.3 Crossover

Crossover is a crucial operation in evolutionary algorithms (EAs) that combines genetic information from two parent solutions to create new offspring solutions. In the context of the traveling salesman problem (TSP), this means combining segments of two tour permutations to generate new offspring tours. Two primary crossover operators are employed in this assignment: edge recombination crossover (ERO) and partially mapped crossover (PMX).

2.3.1 Partially Mapped Crossover (PMX)

In evolutionary algorithms (EAs), crossover plays a pivotal role in combining genetic information from two parent solutions to generate offspring solutions. Partially mapped crossover (PMX) [4] is a constructive crossover operator that specifically targets permutations, such as those used to represent solutions in the traveling salesman problem (TSP).

PMX operates by identifying common subsequences between two parent permutations. These common subsequences represent segments of the tours that are likely to be of high quality, and their preservation can lead to offspring tours that inherit their desirable characteristics. The complete implementation steps of the algorithm are illustrated in Figure 1.

The PMX algorithm involves two main steps:

1. Common Subsequence Identification: Simultaneously traversing the two parent permutations, PMX identifies pairs of consecutive cities that appear in the same order in both permutations. These pairs form common subsequences.
2. Subsequence Extraction and Recombination: The identified common subsequences are extracted from the parents and used to construct offspring permutations. The remaining cities are then inserted into the offspring permutations according to their connectivity relationships.

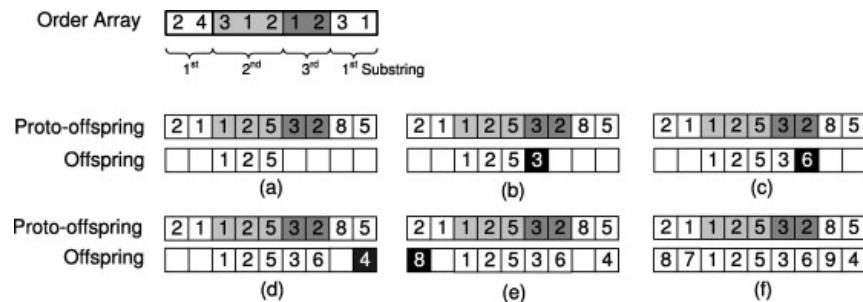


Figure 1: PMX Crossover.

2.3.2 Edge Recombination Crossover (ERO)

Edge recombination crossover (ERO)[5] is a constructive crossover operator that aims to preserve the overall structure of the tours while introducing new genetic material. It operates by identifying a common edge between the two parent tours and then constructing offspring tours by stitching together segments of the parent tours based on this common edge. This approach helps to maintain the connectivity of the tours while introducing new path segments, promoting exploration within the search space.

Overall, ERO is a versatile and effective crossover operator for the TSP, offering a balance between preserving structure and introducing new genetic material. Its ability to enhance exploration and path recombination makes it a valuable tool in EA-based approaches to the TSP. We represent an illustration of edge crossover in Figure 2

Path representation (simplest and most successful)

Genotype: **a e d b c**

Tour: **(a → e → d → b → c)**

Edge recombination crossover

		Edge table											
A:	a-b-c-d-e	⇒	<table><tr><th>City</th><th>has links to</th></tr><tr><td>a</td><td>b, c, e</td></tr><tr><td>b</td><td>a, c, d</td></tr><tr><td>c</td><td>a, b, d, e</td></tr><tr><td>d</td><td>b, c, e</td></tr></table>	City	has links to	a	b, c, e	b	a, c, d	c	a, b, d, e	d	b, c, e
City	has links to												
a	b, c, e												
b	a, c, d												
c	a, b, d, e												
d	b, c, e												
B:	b-d-e-c-a												
AB':	b-c-e-a-d	⇐	<table><tr><td>e</td><td>b, c, d</td></tr></table>	e	b, c, d								
e	b, c, d												

Figure 2: Edge Recombination Crossover.

2.4 Mutation

Mutation is a fundamental operation in evolutionary algorithms (EAs) that introduces random little changes in the population of solutions. This helps to prevent premature convergence and can lead to the discovery of better solutions.

There are two main types of mutation operators utilized in this assignment: insert mutation and swap mutation. Both insert and swap mutations can be effective for solving the TSP. However, insert mutation is more likely to break up good subsequences, while swap mutation is more likely to

preserve the overall structure of the tours. The choice of mutation operator may depend on the specific parameters of the EA and the characteristics of the TSP instance being solved.

In the context of the TSP, mutation is an essential operator that helps to maintain genetic diversity and prevent the EA from becoming stuck in local optima. Both insert mutation and swap mutation can be effective for finding good solutions to the TSP.

2.4.1 Swap Mutation

Swap mutation operates by selecting two random cities in the tour and swapping their positions. This can help to disrupt the current arrangement of the cities and potentially lead to better routes. To implement swap mutation, the following steps are followed and represented in Figure3:

1. Random City Selection: Select two random cities from the current tour.
2. City Swapping: Swap the positions of the two selected cities.

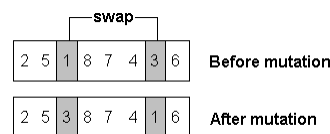


Figure 3: Swap mutation.

2.4.2 Insert Mutation

Insert mutation operates by selecting a random city in the tour and inserting it into a new position. This can help to break up good subsequences in the tours and introduce new possibilities for improving the overall fitness of the solutions.

To implement insert mutation, the following steps are followed and represented in Figure4:

1. Random City Selection: Select a random city from the current tour.
2. Insertion Position Selection: Select a random insertion position for the selected city. The insertion position can be anywhere in the tour, including the current position of the selected city.
3. City Insertion: Insert the selected city into the tour at the selected position. This may involve shifting other cities in the tour to accommodate the insertion.

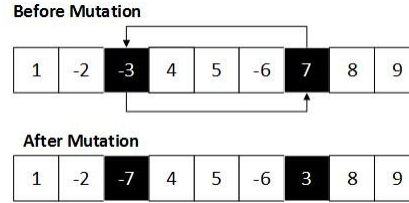


Figure 4: Insert mutation.

2.5 Dataset

In this assignment, we utilized the dataset, *TSPDATA* which provides benchmark data for the Traveling Salesman Problem (TSP). The TSP[6] is a classic NP-hard combinatorial optimization problem that aims to find the shortest tour visiting each city in a given set exactly once [1]. The first line of *TSPDATA.txt* indicates this is a 127-city instance of the TSP. Each subsequent line provides the node number, x-coordinate, and y-coordinate for one of the 127 cities. The edge weights correspond to Euclidean distances between city coordinates. This creates a symmetric TSP instance defined on the Euclidean plane. The moderate size of 127 cities positions this benchmark well for evaluating TSP algorithm performance. In summary, *TSPDATA* provides a standard dataset for an Euclidean 127-city TSP problem that could facilitate TSP research and benchmarking.

3 EXPERIMENTAL RESULTS

In this section of the report, we represent and visualize the results obtained from different combinations of parent or survival selection, crossover, and mutation. As discussed in the previous section, we utilized 2 methods for each EA step. In a nutshell, we examined 8 various combinations of methods:

- Linear ranking + PMX crossover + insert mutation
- Linear ranking + PMX crossover + swap mutation
- Linear ranking + Edge crossover + swap mutation
- Linear ranking + Edge crossover + insert mutation
- FPS + Edge crossover + swap mutation
- FPS + Edge crossover + insert mutation
- FPS + PMX crossover + Swap mutation
- FPS + PMX crossover + insert mutation

3.1 Results

For every combination, we tested an EA for 1000 generations, so that better fitness was achieved. Firstly, we saved the best fitness of each method during a 1000 generation. and then compare them together. According to the fitness function which is designed as below, we focus on maximizing the fitness value. Also for better understanding, fitness value will be divided by 1 since the sum of the route gives us a much better view of what our algorithms are going through. Table classifies the fitnesses we obtained based on the fitness formula for each combination we experimented with. Note that to get better results or the shortest path for the salesman we need to repeat each algorithm e.g. 10 iterations, because of the Random nature of EAs, we obtained various results in one specific algorithm. According to the results gathered in Table 1, we consider the EA with the combination of linear ranking as Parent and survival selection algorithm, PMX crossover, and Swap mutation which is named as LPS with the shortest path (fitness) **235850.41**. Also, the individual with this achieved fitness values is as follows:

83, 116, 78, 17, 23, 10, 89, 63, 57, 12, 56, 50, 1, 21, 20, 16, 18, 59, 58, 61, 88, 91, 98, 124, 112, 64, 99, 113, 105, 14, 13, 29, 33, 42, 34, 49, 120, 53, 123, 54, 90, 60, 115, 75, 74, 87, 109, 70, 2, 119, 107, 31, 28, 27, 126, 92, 117, 45, 111, 110, 106, 97, 96, 122, 93, 46, 48, 43, 44, 102, 80, 82, 81, 100, 101, 62, 26, 30, 25, 32, 121, 86, 85, 84, 103, 65, 55, 52, 47, 94, 39, 36, 0, 114, 6, 104, 22, 72, 73, 68, 118, 95, 108, 69, 67, 71, 9, 51, 4, 5, 8, 7, 66, 125, 79, 77, 15, 35, 40, 41, 38, 24, 37, 11, 19, 3, 76

Table 1: Results of each EA combination based on fitness

	Edge		PMX	
	Swap	Insert	Swap	Insert
Linear Ranking	274747.06	269329.71	235850.41	244519.63
FPS	512298.82	502278.73	531528.64	531832.64

In Table 2, The time it takes to complete each EA combination is represented to understand how the algorithms are fast to do the operations for 1000 generations. appeared that the best combination with the best fitness has the lowest spent time which makes it suitable for choosing as the optimal EA.

Table 2: Duration (Seconds) takes to complete each EAs.

	Edge		PMX	
	Swap	Insert	Swap	Insert
Linear Ranking	778.34	805.36	162.62	183.82
FPS	760.44	760.01	190.83	192.58

3.2 Experiments

Always visualizing gave us a better view of how the algorithms perform during an iteration. So in Figure 5, We demonstrate the effectiveness of an evolutionary algorithm (EA) in solving the traveling salesman problem (TSP) for 1000 generations. The EA employed a combination of linear ranking selection, edge recombination crossover, and insert mutation to navigate the search space and identify promising routes. The EA's performance consistently improved over time, as evidenced by the decrease in the best fitness solution found in each generation.

As evident, for the algorithms which utilized Linear ranking for parent and survival selection, We observe a regular reduction in the size of the routes. but for the ones that used FPS for the parent and survival selection too many swings were observed in related graphs. Note that for each combination we extracted the best fitness of the best route so we conclude that the visualizing is just for understanding the algorithm performance.

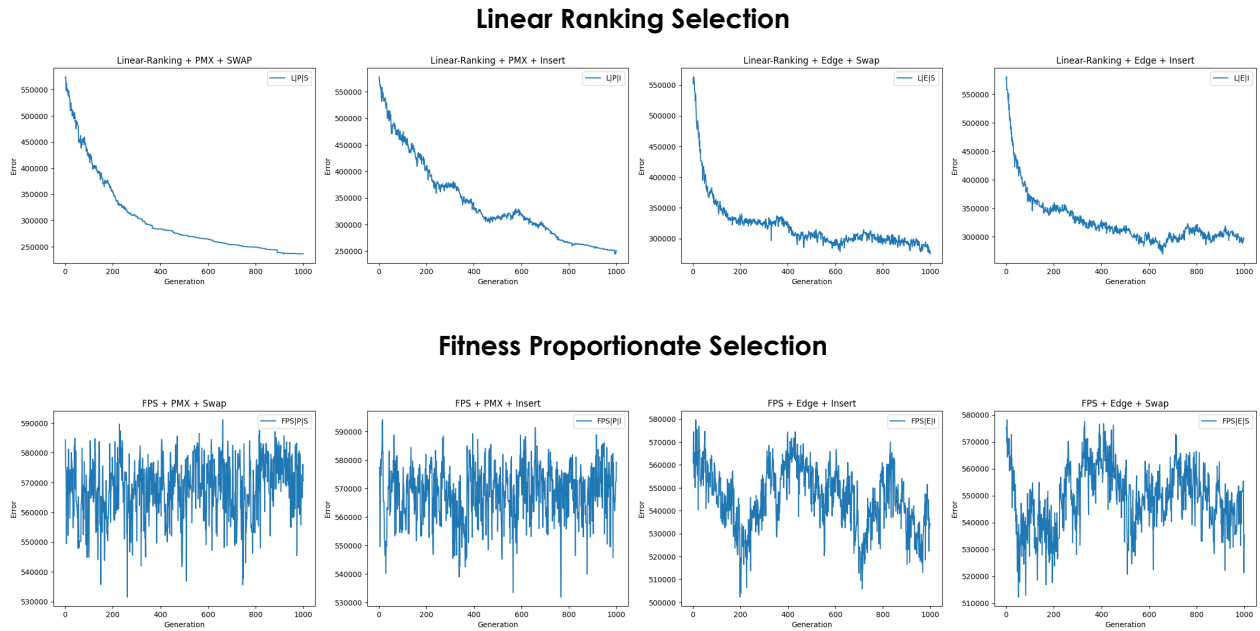


Figure 5: Visualizing the process of error reduction (shortest path) in 1000 generations.

4 CONCLUSION

In this study, various image processing techniques were explored including image zooming, contrast enhancement, histogram equalization, and geometric transformations. For image zooming, pixel replication and zero-order hold methods were implemented with a zoom factor of 2. It was found that pixel replication leads to pixelation while zero-order hold preserves more details. However, increasing the zoom factor deteriorates image quality rapidly for both methods. Contrast enhancement through logarithmic and gamma corrections improved the visibility of details in a low-contrast image. Histogram equalization redistributes pixel intensities to enhance contrast globally. Translating and rotating geometric shapes in an image was achieved through affine transformations. Finally, pixel-wise operations like thresholding and blurring were applied to modify image appearance.

Overall, this exercise provided essential hands-on experience in implementing fundamental image processing operations in OpenCV and analyzing their effects. Key skills developed include normalizing pixel values, plotting histograms, applying point and neighborhood processing, and performing geometric transformations. The limitations of simple methods like zooming and contrast stretching also became apparent. Advanced techniques would be required for zoom factors higher than 2 without compromising details. Similarly, adaptive contrast enhancement adapts to local variations rather than uniformly stretching contrast.

Through this exploration, the building blocks of image processing algorithms were understood. The importance of tasks like normalization and histogram analysis was realized. Transforming images geometrically and modifying pixel values locally opened up avenues for more sophisticated image manipulation. Going forward, combining these basic operations into pipelines and studying state-of-the-art techniques will help develop proficiency in addressing real-world image processing challenges. The hands-on experience from this exercise established strong foundations in terms of concepts, skills, and implementation.

In summary, this assignment served as an engaging introduction to image processing, equipping with key concepts and practices. The experience gained will pave the way for taking on advanced techniques and complex image manipulation challenges. The knowledge and skills will be invaluable for any work involving analyzing and enhancing images automatically.

REFERENCES

- [1] M. Chakraborty and U. K. Chakraborty, "Branching process analysis of linear ranking and binary tournament selection in genetic algorithms," *Journal of computing and information technology*, vol. 7, no. 2, pp. 107–113, 1999.
- [2] J. Arabas, Ł. Bartnik, and K. Opara, "Dmea—an algorithm that combines differential mutation with the fitness proportionate selection," in *2011 IEEE Symposium on Differential Evolution (SDE)*, IEEE, 2011, pp. 1–8.
- [3] F. Yu, X. Fu, H. Li, and G. Dong, "Improved roulette wheel selection-based genetic algorithm for tsp," in *2016 International conference on network and information systems for computers (ICNISC)*, IEEE, 2016, pp. 151–154.
- [4] B. Koohestani, "A crossover operator for improving the efficiency of permutation-based genetic algorithms," *Expert Systems with Applications*, vol. 151, p. 113381, 2020.
- [5] A. J. Umbarkar and P. D. Sheth, "Crossover operators in genetic algorithms: A review.," *ICTACT journal on soft computing*, vol. 6, no. 1, 2015.
- [6] N. M. Razali, J. Geraghty, *et al.*, "Genetic algorithm performance with different selection strategies in solving tsp," in *Proceedings of the world congress on engineering*, International Association of Engineers Hong Kong, China, vol. 2, 2011, pp. 1–6.